

实验 2：工业级高并发线程池实现（学生版）

1. 实验目标

- 深入理解池化技术：**理解线程池如何通过复用线程减少 OS 开销。
- 掌握 POSIX 同步原语：**熟练掌握互斥锁（Mutex）与条件变量（Condition Variable）的组合用法。
- 处理工程细节：**解决虚假唤醒（Spurious Wakeup）及优雅退出（Graceful Shutdown）等工业级问题。

2. 实验任务

任务一：补全工作线程 (Worker) 逻辑

工作线程是线程池的核心。它必须在一个无限循环中：

- 等待任务到达（需处理虚假唤醒）。
- 判断线程池是否正在关闭。
- 从链表头部提取任务并更新队列。

思考与报告要求：

- 解释为什么 `pthread_cond_wait` 必须放在 `while` 循环中，而不能只用 `if` 判断。
- 说明为什么取出任务后要先释放锁，再执行 `task->function(task->argument)`。

任务二：补全任务提交 (Submit) 逻辑

主线程（或生产者）调用此接口。它需要：

- 封装任务节点。
- 安全地将节点插入链表尾部。
- 通知（Signal）其中一个空闲的 Worker 线程。

思考与报告要求：

- 说明为什么提交任务时需要在持锁状态下检查 `shutdown` 标志。
- 说明为什么提交一个任务后使用 `pthread_cond_signal`，而不是默认使用 `pthread_cond_broadcast`。

任务三：实现优雅关闭 (Destroy) 逻辑

- 修改 `shutdown` 标志位。
- 广播 (Broadcast)** 唤醒所有正在休眠的线程。
- 等待 (Join) 所有线程安全退出。

思考与报告要求：

- 说明为什么关闭线程池时需要使用 `pthread_cond_broadcast` 唤醒所有工作线程。
- 说明为什么 `thread_pool_destroy` 中应先释放互斥锁，再调用 `pthread_join` 等待工作线程退出。

3. 框架代码说明

3.1 代码框架 (threadpool.c)

你需要在以下代码块中补全逻辑。请特别注意**锁的持有时间**。

下面的框架代码中已经包含了用于基础运行验证的 `main` 函数。你只需要补全各处 `TODO`，即可直接编译运行。

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

// 任务结构体
typedef struct Task {
    void (*function)(void *);
    void *argument;
    struct Task *next;
} Task;

// 线程池结构体
typedef struct ThreadPool {
    pthread_mutex_t lock;
    pthread_cond_t notify;
    pthread_t *threads;
    Task *task_queue_head;
    Task *task_queue_tail;
    int thread_count;
    int shutdown;
} ThreadPool;

// 工作线程逻辑
void *thread_pool_worker(void *arg) {
    ThreadPool *pool = (ThreadPool *)arg;

    while (1) {
        pthread_mutex_lock(&pool->lock);

        // TODO 1: 处理虚假唤醒。当队列为空且池未关闭时，线程应在此休眠。
        // 提示: 使用 pthread_cond_wait

        // TODO 2: 检查退出条件。
        // 如果池正在关闭且队列已空，则释放锁并退出线程 (pthread_exit)

        // TODO 3: 获取任务。从 task_queue_head 取出一个任务并更新头尾指针。
        Task *task = NULL;
        // 你的代码...

        pthread_mutex_unlock(&pool->lock);

        // 执行任务
        if (task != NULL) {
            task->function(task->argument);
        }
    }
}
```

```

        free(task);
    }
}
return NULL;
}

// 提交任务
int thread_pool_submit(ThreadPool *pool, void (*function)(void *), void *argument) {
    // TODO 4: 封装任务并插入队列尾部。注意对 pool->lock 的保护。
    // 记得调用 pthread_cond_signal 唤醒线程。
    return 0; // 成功返回 0
}

// 销毁线程池
void thread_pool_destroy(ThreadPool *pool) {
    // TODO 5: 实现优雅退出逻辑。设置 shutdown 标志，唤醒所有线程，并 join 它们。
    // 最后释放互斥锁、条件变量及内存。
}

// ===== 测试任务 =====
void my_task(void *arg) {
    int id = *(int *)arg;
    printf("Thread %lu 正在执行任务 %d...\n", (unsigned long)pthread_self(), id);
    usleep(100000); // 模拟耗时 0.1 秒的任务
    free(arg);      // 释放参数内存
}

// ===== main: 基础运行验证入口 =====
int main() {
    ThreadPool *pool = (ThreadPool *)malloc(sizeof(ThreadPool));
    if (pool == NULL) {
        return 1;
    }

    pool->thread_count = 4;
    pool->shutdown = 0;
    pool->task_queue_head = NULL;
    pool->task_queue_tail = NULL;

    pthread_mutex_init(&pool->lock, NULL);
    pthread_cond_init(&pool->notify, NULL);

    pool->threads = (pthread_t *)malloc(sizeof(pthread_t) * pool->thread_count);
    if (pool->threads == NULL) {
        pthread_mutex_destroy(&pool->lock);
        pthread_cond_destroy(&pool->notify);
        free(pool);
        return 1;
    }

    for (int i = 0; i < pool->thread_count; i++) {
        pthread_create(&pool->threads[i], NULL, thread_pool_worker, (void *)pool);
    }
}

```

```
printf("=== 线程池启动, 投入 15 个任务 ===\n");
for (int i = 1; i <= 15; i++) {
    int *arg = (int *)malloc(sizeof(int));
    *arg = i;
    thread_pool_submit(pool, my_task, arg);
}

thread_pool_destroy(pool);
return 0;
}
```

4. 基础运行与验证

在完成代码补全后, 先编译:

```
gcc -Wall -o thread_pool threadpool.c -lpthread
```

然后运行:

```
./thread_pool
```

`./thread_pool` 用于验证线程池是否能正确执行任务并完成优雅退出。这属于本实验的基础要求。

5. 选做挑战 (Bonus)

挑战 A: Future/Promise 模式

当前任务是“发后即忘 (Fire and Forget)”。请尝试修改接口, 使得调用者可以阻塞等待任务的执行结果。

挑战 B: 使用 Valgrind 做资源释放检查

在基础功能已经跑通后, 可进一步使用:

```
valgrind --leak-check=full ./thread_pool
```

检查你的实现是否真正释放了所有资源。

这不是新的功能开发任务, 而是对 `destroy`、`submit` 失败路径以及任务节点释放逻辑的高标准验证。

如果 `valgrind` 仍报告泄漏或挂起资源, 请根据输出结果回查:

- 是否正确 `join` 了所有工作线程
 - 是否释放了 `pool->threads` 与 `pool`
 - 是否在关闭状态下正确回收了未提交成功的任务节点
-

6. 实验提交

完成实验报告的填写，以“学号-姓名-操作系统实验二”命名

将实验报告提交到作业网站: <https://acm.scu.edu.cn/teach/>